Chapter 1 Experimental Results

This section presents the experimental validation of the proposed rApp, which covers three aspects: (1) packet timing correction, (2) configuration mapping verification, and (3) automation versus manual process comparison.

1.1 Test Environment

The experiments were conducted on a high-performance server equipped with the hardware and software specifications listed in Tables 1.1–1.3.

Table 1.1: Hardware Environment

Item	**Info**
CPU	Intel® Xeon® Gold 6433N [**?**]
CPU Product Collection	4th Gen Intel® Xeon® Scalable Processors [**?**]
CPU Total Cores	32
CPU Total Threads	64
Max Turbo Frequency	3.60 GHz
Base Frequency	2.00 GHz
NUMA Node	NUMA node(s): 1, node0 CPU(s): 0–31
Memory	256 GB (64 GB × 4)
Disk	500 GB
NIC	Intel XXV710 for 25GbE SFP28 [**?**]
Server Model	Supermicro SYS-521E-WR [**?**]

Table 1.2: Software Environment

Item	**Info**
Operating System	Red Hat Enterprise Linux 9.2 (Plow) [**?**]
Kernel Version	5.14.0-284.30.1.rt14.315.el9_2.x86_64 [**?**]
DPDK Version	20.11.9 [**?**]
LinuxPTP Version	3.1.1 [**?**]
OAI Branch	develop

Table 1.3: Radio Unit (RU) Information

Item	**Info**
RU Product Name	JURA_RU
Firmware Version	2.2.8

Table 1.4: PTP config

Item	**Info**
PTP version	IEEE 1588v2
PTP profile	G8275.1

1.2 Experiment I: Packet Timing Adjustment

To verify whether the transmission timing between the Distributed Unit (DU) and Radio Unit (RU) can be adjusted from `early`/`late` states to `on-time`, we conducted tests across three different topologies: short fiber (C3), short fiber (C1), and long fiber (C1), as shown in Figure 1.1.

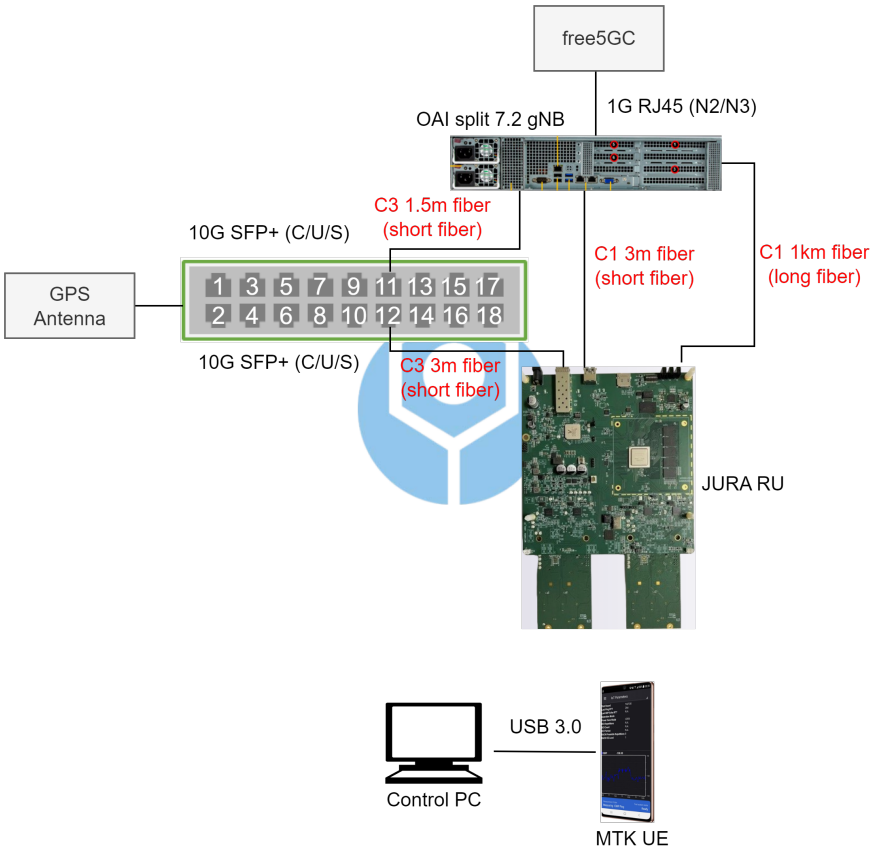

Figure 1.1: Testbed for timing window adjustment experiments under different fiber topologies.

In these experiments, the proposed rApp dynamically adjusted the RU timing window by analyzing the delay trend observed through the synchronization status of the fronthaul packets. After applying rApp-based tuning, the timing alignment was significantly improved across both C-plane and U-plane traffic.

Figure 1.2 presents the `delta_RX_EARLY` values before and after timing adjustment. Across all topologies, the early packet counts were dramatically reduced, indicating that packets originally arriving too early were successfully shifted to the correct receive window.

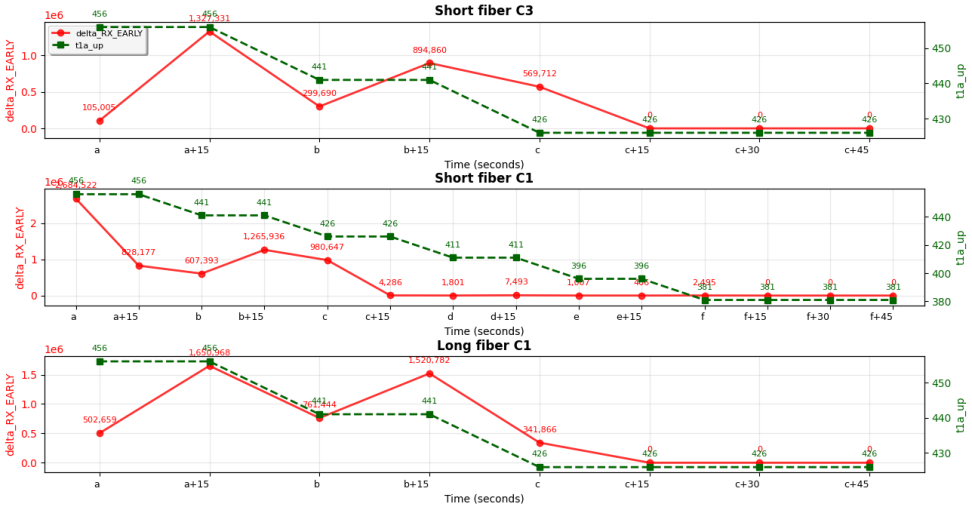

Figure 1.2: Timing correction results: `delta_RX_EARLY` across different topologies.

Similarly, Figure 1.3 shows the `delta_RX_LATE` values, where the number of late-arriving packets decreased to nearly zero after correction. This confirms the rApp's ability to mitigate both early and late packet alignment issues and drive the system toward an on-time transmission state.

1.3 Experiment II: Configuration Mapping Verification

This experiment aims to validate whether the configuration parameters assigned to the DU and RU are correctly applied through the proposed

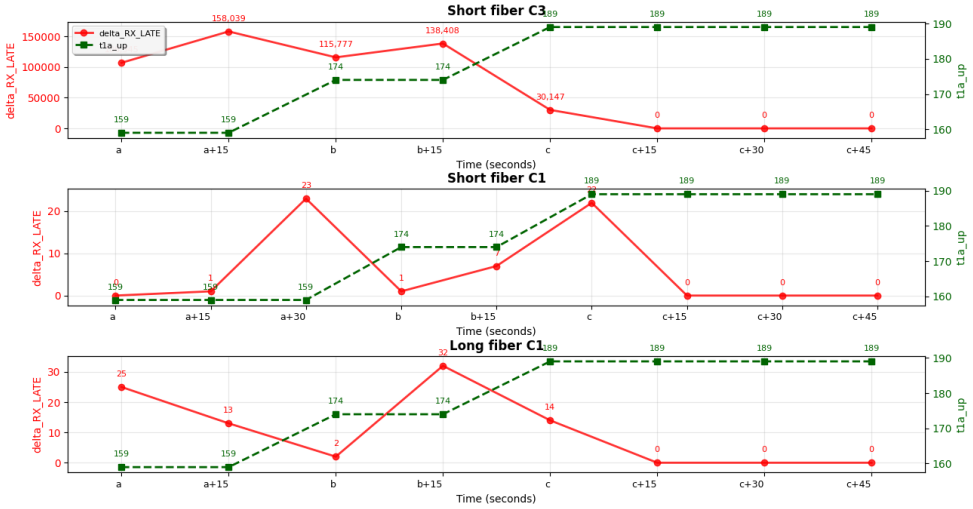

Figure 1.3: Timing correction results: `delta_RX_LATE` across different topologies.

rApp. Two representative use cases were tested: bandwidth (BW) adjustment and MIMO configuration adjustment. The testbed setup is shown in Figure 1.4.

We define the configuration verification logic as follows:

Throughput (THPUT) achieved ⇒ UE successfully attached to gNB ⇒ Configuration ap

To ensure consistency across tests, the `iperf3` throughput evaluation tool was configured with fixed parameters as shown in Table 1.5.

Table 1.5: iperf throughput test configuration

Test case Parameters	**Value**
Target Downlink Throughput	700 Mbps
Target Uplink Throughput	100 Mbps
Test Duration	5 minutes

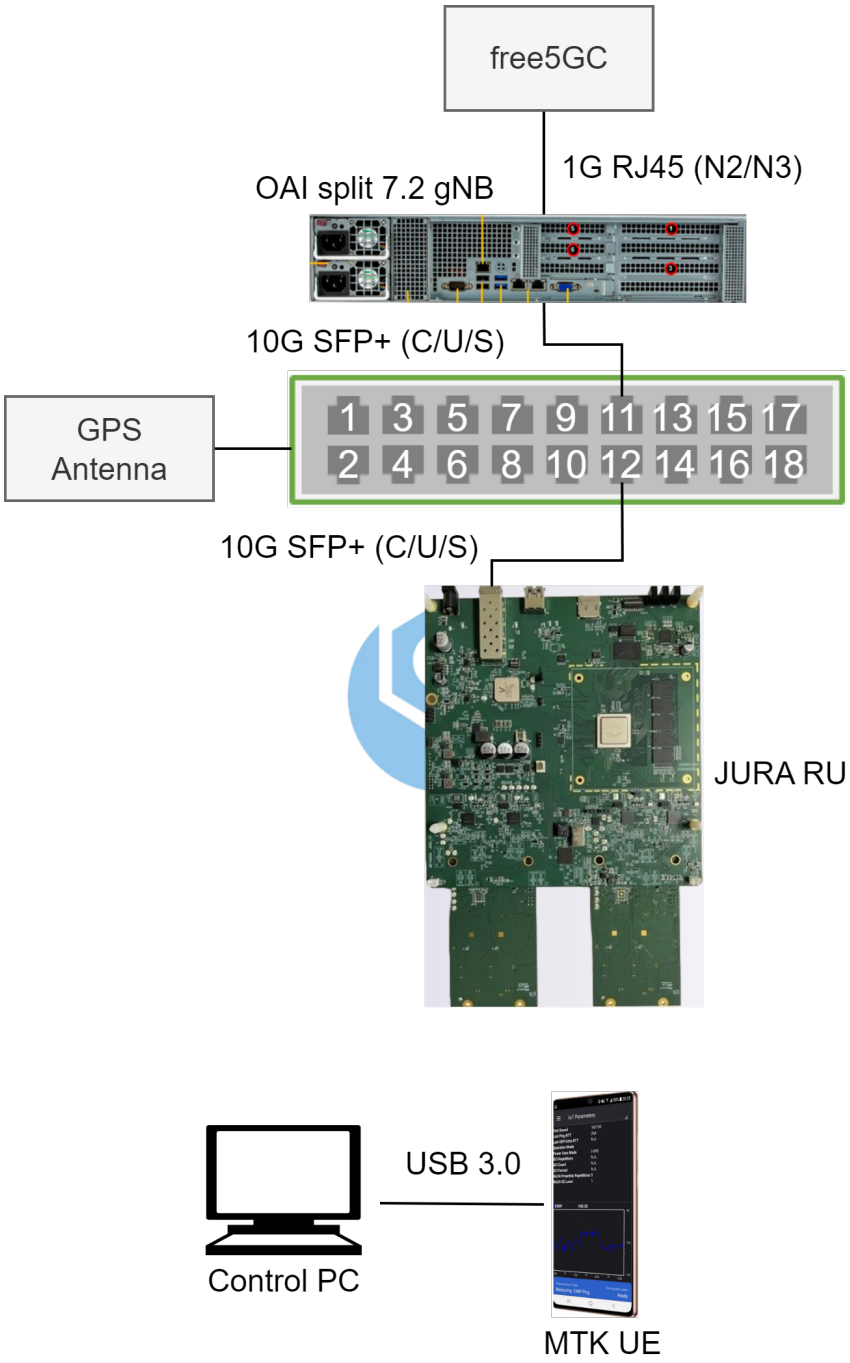

Figure 1.4: Testbed for configuration mapping and bandwidth variation experiments.

Use Case 1: Bandwidth Adjustment

Four different bandwidth settings were applied: 20, 40, 60, and 100 MHz. Table 1.6 shows the associated frequency configurations, including the `arfcn` and `bSChannelBw`.

Table 1.6: Bandwidth experiment configuration

Test case Parameters	**Test1**	**Test2**	**Test3**	**Test4**
ARFCN	640596	640008	646724	646724
bSChannelBw (MHz)	20	40	60	100

As shown in Figure 1.5, the uplink throughput increased in proportion to the configured bandwidth, verifying the correctness of bandwidth-related parameter mapping between the DU and RU.

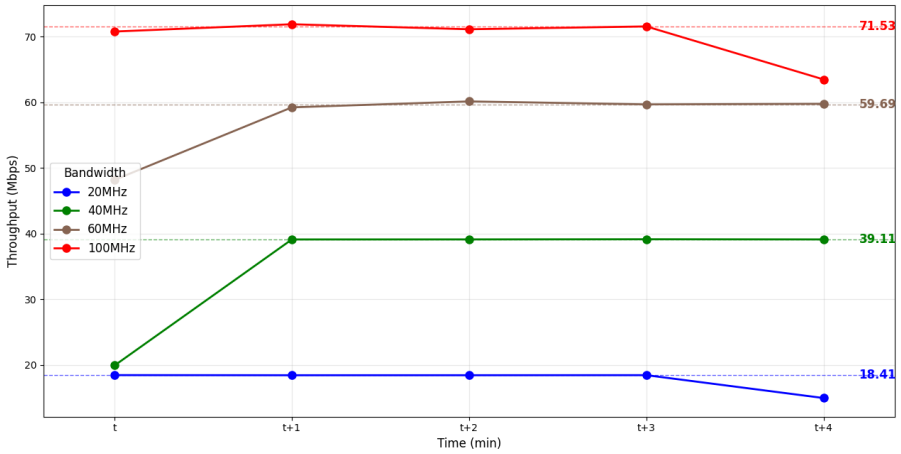

Figure 1.5: Uplink throughput under different bandwidth configurations.

Use Case 2: MIMO Adjustment

In this scenario, six different MIMO configurations were tested by adjusting both the number of antennas (2x2 and 4x4) and the number of MIMO layers (from 1 to 4). The detailed configuration is shown in Table 1.7.

Table 1.7: MIMO experiment configuration

Test case Parameters	Test1	Test2	Test3	Test4	Test5	Test6
Antenna Configuration	2x2	2x2	4x4	4x4	4x4	4x4
MIMO Layers	1	2	1	2	3	4

The results in Figure 1.6 demonstrate that the downlink throughput scales with the number of configured MIMO layers, confirming that MIMO-related parameters were successfully mapped and applied.

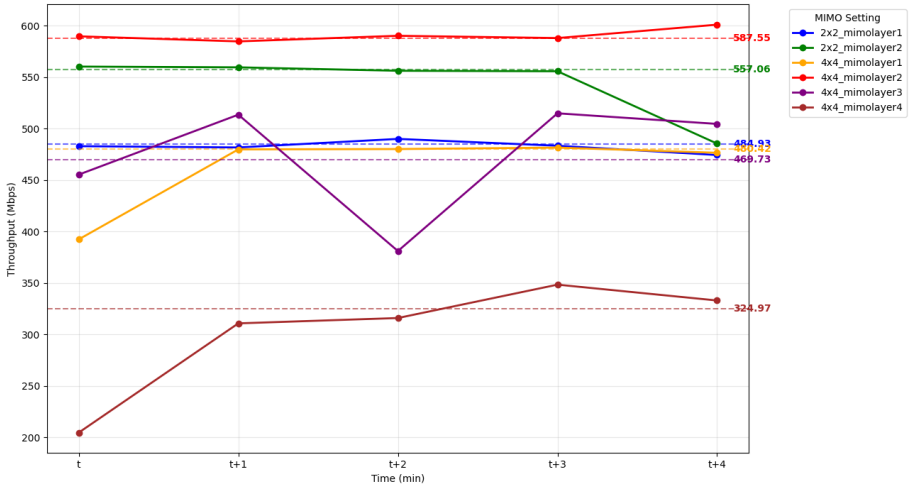

Figure 1.6: Downlink throughput under different MIMO configurations.

To further support the correctness of configuration mapping, the RU's energy consumption was monitored. As shown in Figure 1.7, RU power usage also increased with higher MIMO complexity, which is consistent

with the expected workload for increased data transmission capacity.

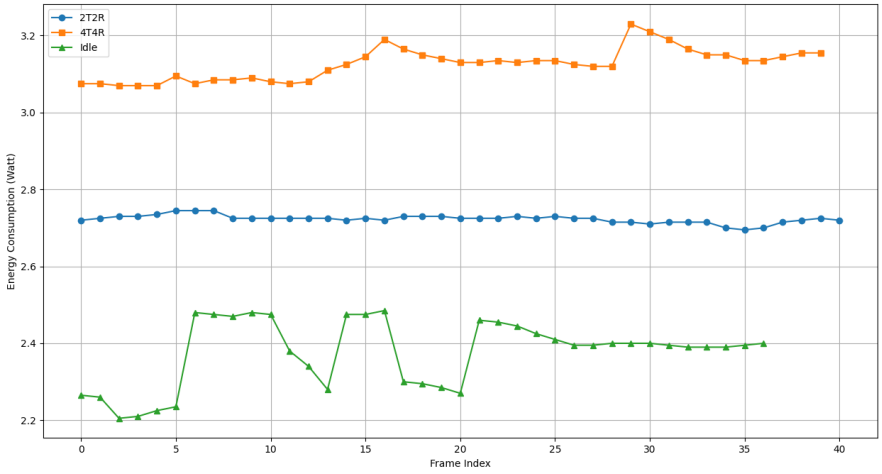

Figure 1.7: Energy consumption of JURA RU under different MIMO configurations.

1.4 Experiment III: Automation vs. Manual Process Comparison

This experiment aims to evaluate the effectiveness of the proposed rApp in reducing configuration complexity and execution time compared to manual operation. Two key metrics were used for comparison:

- **Configuration complexity**, measured by the number of parameters and command-line operations required.
- **Execution time**, measured by the total time taken to complete one full end-to-end test process.

In the automated scenario, all rApp commands were timestamped by

the system log to measure the time between command invocation and completion. In the manual scenario, we used a stopwatch to record the time needed to perform each step manually.

Metric 1: Configuration Complexity

Figure 1.8 shows the comparison of required configuration effort. This includes the number of parameters set, commands executed, and device terminals accessed. Automation via the rApp significantly reduced the complexity, allowing operators to perform the same procedures with fewer steps and lower risk of manual error.

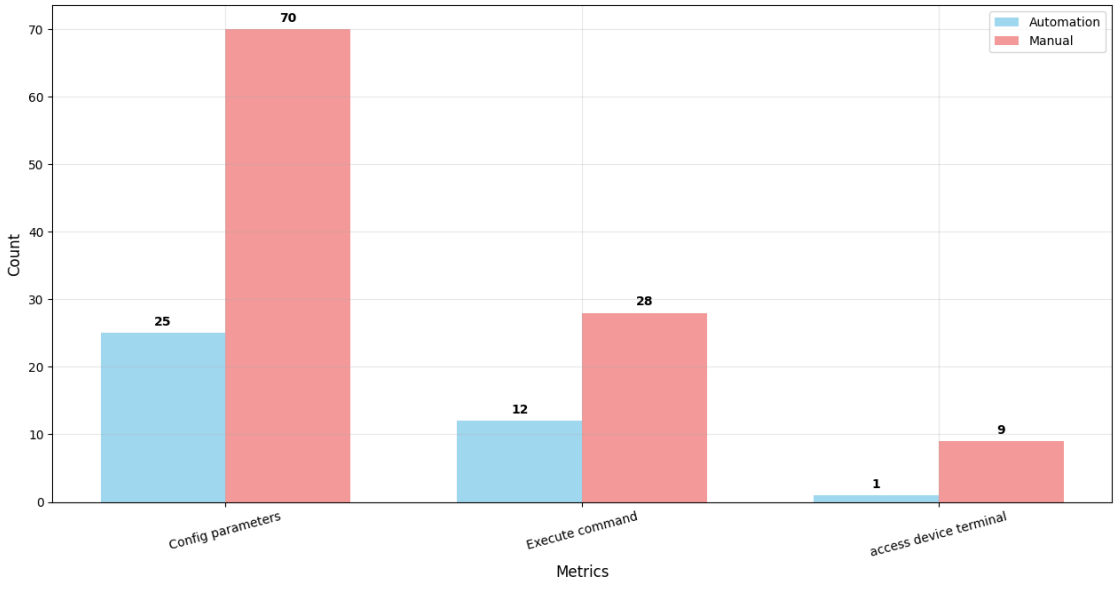

Figure 1.8: Comparison of required configuration effort: automation vs. manual process.

Metric 2: Execution Time

To assess time savings, we measured the duration of each integration step—such as DU/RU configuration, SMO registration, timing window adjustment, and UE attachment—in both manual and automated workflows. As illustrated in Figure 1.9, automation reduced the total execution time from 1218 seconds to 363 seconds, representing a 70% improvement in overall efficiency.

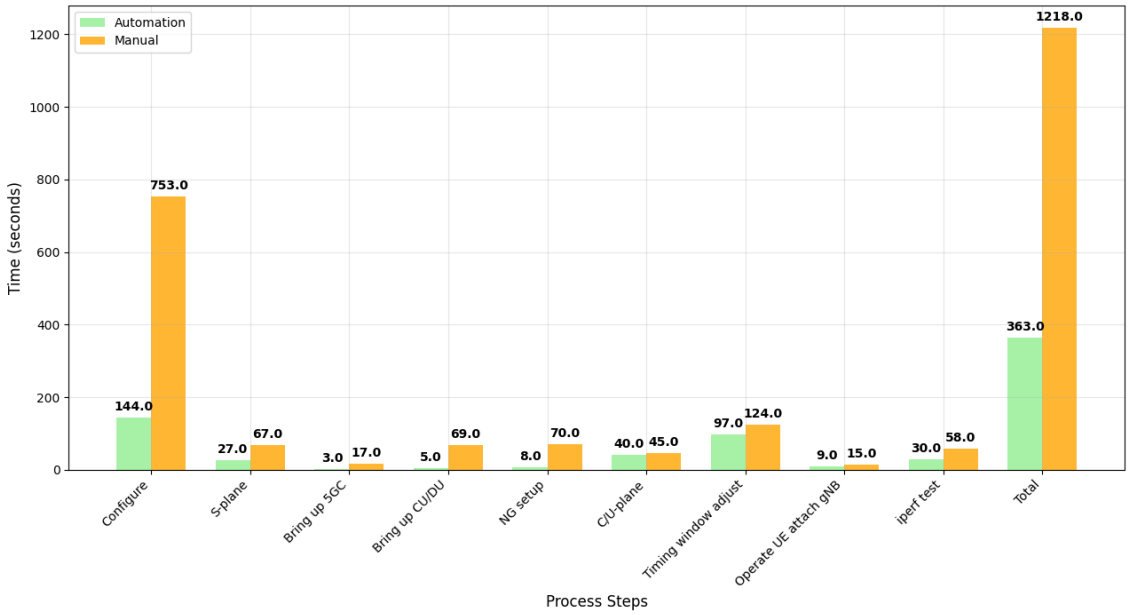

Figure 1.9: Comparison of execution time for each integration step: automation vs. manual process.

These results demonstrate that the proposed rApp not only enables correct and dynamic configuration of DU and RU, but also significantly improves testing efficiency by reducing both manual effort and time consumption.